Ingeniería del Software II

Práctica 3: pruebas y depuración de la mascota

¿Podemos entregar esta versión?

Rocío Carratalá Sáez (L1, L2), Inmaculada García Pereira (L3)

Grado en Ingeniería Informática; Doble Grado en Matemáticas e Ingeniería Informática
Universitat de València

Curso 2026–27

Objetivo y punto de partida

Investigaréis una versión de la mascota de PR2 con defectos y pruebas iniciales insuficientes. Diseñaréis casos, automatizaréis pruebas y corregiréis un defecto con evidencia. Al terminar, decidiréis si la versión puede entregarse.

Material de partida. La profesora facilitará el proyecto `mascota-pruebas` con las funcionalidades implementadas, cuatro pruebas iniciales y los ejercicios de caja blanca y mutación. El entorno Java, VS Code y Maven Wrapper será el utilizado en PR2.

Entrega

Para esta práctica, se entregará/actualizará:

- Jira: del mismo modo que en la PR2, en el mismo espacio de la PR1, pero creando una **nueva *epic***, deben reflejarse las distintas **tareas** y **quién** las ha llevado a cabo, así como los **bloqueos**. Para realizar estas tareas cread un **nuevo sprint** que dure tanto como el plazo para realizar y entregar esta práctica.
- Memoria: a través del **Aula Virtual**, con el contenido detallado a lo largo de este enunciado.
- GitHub: a lo largo del enunciado se indicará en qué momentos **debe actualizarse el repositorio** para compartir versiones actualizadas del código o ficheros de salida. El repositorio lo habrá creado la profesora (del mismo modo que para la práctica anterior) antes de iniciar esta práctica. **Cada estudiante debe trabajar en una rama independiente** y, finalmente, estas deben integrarse en la rama principal mediante *pull requests*.
  Recordad que los compilados no deben subirse al repositorio de GitHub. Por tanto, los cambios sobre `target/` no se deben preparar ni añadir. Puede optarse por borrar dicho directorio mediante `rm -rf target/`.

1. Organizar el trabajo

Quién lo hace: todo el grupo.

El alcance, las técnicas y el orden de las pruebas ya están definidos en este enunciado. Revisad la propuesta de reparto: sustituid las letras por los nombres y apellidos de los integrantes del grupo. Si fuera necesario, corregid cualquier asignación que resulte incompatible. Memoria: tras las asignaciones, completad la siguiente tabla y añadidla a la memoria:

Apartado	Responsable	Revisor/a
2. Diseño de caja negra	A	B
3. Análisis de caja blanca	B	C
4. Automatización con JUnit	C	D
5. Pruebas de integración	C	D
6. Depuración y regresión	D	E
7. Mutación	E	A
8. Aceptación y conclusión	Todos/as	Todos/as

La persona responsable coordina la ejecución y registra las evidencias. Otra persona revisa los resultados frente a lo solicitado.

2. Pruebas de caja negra

Trabajad en este apartado sin consultar la implementación de los métodos que vais a probar (se trata de pruebas de caja negra). La revisión de estas reglas es una actividad estática; la ejecución posterior de los casos será dinámica.

Especificación del estado de la mascota y de cada funcionalidad

- **Estado.** Hambre, energía y felicidad son enteros entre 0 y 100. Preparad estados concretos con el constructor de cuatro argumentos: `new Mascota("Luna", hambre, energia, felicidad)`. Los casos independientes usan objetos nuevos; una secuencia conserva la misma mascota.
- **Jugar.** Solo se permite si hambre ≤ 80 **y** energía ≥ 20. Una acción válida aumenta el hambre en 10, reduce la energía en 20 y aumenta la felicidad en 15. Los niveles se limitan al intervalo 0–100. No cambia el nombre, el color ni el contador de comidas.
- **Rechazo.** Si no se puede jugar, se lanza `IllegalArgumentException` antes de modificar la mascota. No se añade ningún evento al historial.
- **Descansar.** Si la energía es menor que 100, aumenta en 30 y el hambre aumenta en 5, con saturación en 100. Se conserva la felicidad. Con energía 100 se rechaza sin cambios.
- **Nombre.** Se eliminan espacios de los extremos. El resultado debe contener entre 1 y 20 caracteres. Un nombre nulo, vacío, compuesto solo por espacios o demasiado largo se rechaza sin modificar el anterior. Para medir longitudes en esta práctica, usad letras ASCII.
- **Historial.** Los servicios modifican la mascota; el controlador registra exactamente un evento después de cada acción correcta. El evento contiene el estado posterior. Una llamada directa a un servicio no registra eventos.

Casos que debéis diseñar

Las cuatro tablas siguientes fijan los datos de entrada. Completad las decisiones y expectativas aplicando: **tabla de decisión** (cuatro combinaciones de las condiciones de juego), **valores límite** (hambre 79/80/81 y energía 19/20/21), **particiones del nombre** (incluidos nulidad, espacios y límites de longitud) y **transiciones** (tres intentos de juego y descanso sobre la misma mascota).

Memoria: entregad las cuatro tablas completadas, vuestro ejecutor Java, los comandos utilizados y el registro completo de las 23 filas. Relacionad cada salida con su ID e indicad la versión probada y los cambios locales.

Formato de las celdas. En esperado y observado, escribid aceptación/rechazo y estado final H/E/F. En veredicto, usad **C** (coincide) o **F** (falla); si no se pudo ejecutar, **NE** y la causa. Que Java termine sin error no demuestra que el comportamiento sea correcto. En todos los casos, comprobad también nombre, color y comidas: deben conservarse al jugar. Anotad cualquier cambio indebido.

Tabla 1. Decisiones. H/E/F significa hambre/energía/felicidad. Completad C1 ($H \leq 80$), C2 ($E \geq 20$) y las tres últimas columnas antes/después de ejecutar, según corresponda.

ID	Inicial	C1	C2	Esperado	Observado	Veredicto
D1	50/50/50					
D2	81/50/50					
D3	50/19/50					
D4	81/19/50					

Tabla 2. Valores límite. La acción es jugar. El motivo de selección aparece en la columna frontera. Aunque L3 y L4 repiten entradas de D2 y D3, vuestro ejecutor debe identificar ambas filas para facilitar la trazabilidad.

ID	Inicial	Frontera	Esperado	Observado	Veredicto
L1	79/50/50	H: anterior			
L2	80/50/50	H: límite			
L3	81/50/50	H: posterior			
L4	50/19/50	E: anterior			
L5	50/20/50	E: límite			
L6	50/21/50	E: posterior			
L7	80/20/50	Ambos límites			

Observaciones: identificad las filas que incumplen el contrato y explicad la discrepancia concreta. Conservad los resultados de la versión inicial aunque más adelante se corrija el programa.

Tabla 3. Particiones y límites del nombre. Cada caso parte de Luna y 50/50/50. Escribid aceptación/rechazo y nombre final en esperado y observado. Los niveles, color y comidas deben conservarse; anotad cualquier cambio. Mostrad la entrada entre corchetes para distinguir espacios y cadena vacía; los corchetes no forman parte del nombre.

ID	Entrada / motivo	Esperado	Observado	Veredicto
N1	`null`: ausencia de nombre			
N2	Cadena vacía: longitud 0			
N3	Tres espacios: nombre en blanco			
N4	`a`: longitud 1, válida			
N5	20 letras `a`: límite superior			
N6	21 letras `a`: exceso			
N7	Nube con un espacio a cada extremo: recorte			
N8	20 letras `a` con un espacio a cada extremo: recorte y límite			

Tabla 4. Transiciones sobre una misma mascota. Inicio: Luna, 50/40/50. En esperado y observado, anotad aceptación/rechazo, H/E/F final y si permite jugar después. Registrad el resultado de `puedeJugar()` después de cada paso y contrastadlo también con el contrato.

ID	Acción	Esperado	Observado	Veredicto
T1	Primer juego			
T2	Segundo juego			
T3	Tercer intento			
T4	Descanso			

Si un paso se desvía de lo esperado, registrad también los pasos posteriores con el estado real: no reiniciéis la mascota ni copiéis los estados teóricos como si fueran observados. Indicad la primera discrepancia y qué efectos posteriores dependen de ella. Un rechazo previsto puede ser correcto; un rechazo no previsto es un fallo.

Implementar y ejecutar vuestras pruebas de caja negra

Implementad vosotros el ejecutor de las 23 filas: D1–D4, L1–L7, N1–N8 y T1–T4. Esto debe hacerse sin consultar/conocer la implementación interna de la mascota ni modificar sus reglas.

Archivo y estructura. Cread `PruebasCajaNegra.java` en `src/main/java/es/uv/is2/mascota/`, con el paquete `es.uv.is2.mascota` y un método `public static void main(String[] args)`. Es un programa Java de pruebas, sin JUnit. Podéis organizar los datos en arrays y recorrerlos con bucles; no hace falta repetir todo el código para cada fila ni construir un menú.

Interfaz pública que podéis utilizar:

- Preparación: `new Mascota("Luna", hambre, energia, felicidad)`.
- Juego: construid `new ServicioJuego(mascota)` y llamad a `jugar()`.
- Nombre: construid `new ServicioAspecto(mascota)` y llamad a `renombrar(entrada)`.
- Descanso: construid `new ServicioDescanso(mascota)` y llamad a `descansar()`.
- Observación del estado: usad los métodos `getNombre()`, `getHambre()`, `getEnergia()`, `getFelicidad()`, `getColor()` y `getComidas()`. Consultad también `puedeJugar()` sobre la mascota.

Cómo implementar cada caso independiente (D, L y N):

1. Escribid su ID y entrada. Cread una mascota nueva con el estado de la tabla y guardad o imprimid todos sus valores anteriores.
2. Ejecutad **una única acción** sobre el servicio correspondiente, dentro de un `try`. Si retorna normalmente, registrad "aceptada". Capturad `IllegalArgumentException` y registrad "rechazada", el tipo de excepción y el mensaje. No capturéis indiscriminadamente cualquier error como si fuera un rechazo válido.
3. Tras el `try/catch`, imprimid el estado final, también cuando haya rechazo. Así podréis comprobar que se conservan los campos que no debían cambiar. Continuad con el siguiente caso: un rechazo previsto no debe terminar el programa.
4. Para N1 pasad `null` de Java, no el texto `"null"`. Diferenciad la cadena vacía de una cadena con tres espacios. Podéis generar veinte letras con `."a".repeat(20)`. Añadid los espacios exteriores en N7 y N8 sin recortarlos en vuestro ejecutor: esa responsabilidad pertenece al servicio probado.

Cómo implementar T1–T4. Cread una sola mascota con 50/40/50 antes de la secuencia. Ejecutad tres llamadas a jugar y una a descansar sobre ese mismo objeto. Aplicad el registro y la captura de excepciones a **cada paso por separado**. Después de cada paso registrad también `puedeJugar()`. Continuad desde el estado real, sin recrear ni ajustar la mascota para forzar el estado esperado.

GitHub: debe actualizarse para incorporar las implementaciones de este apartado.

Qué debéis comprobar. Vuestro ejecutor imprime observaciones. Comparadlas con las expectativas calculadas antes, completad las cuatro tablas y justificad C/F/NE.

Formato mínimo del registro de cada caso/paso:

```
CASO <ID> ACCION <accion> ENTRADA <datos>
ANTES nombre=... hambre=... energia=... felicidad=... color=... comidas=...
RESULTADO <aceptada o rechazada; tipo y mensaje si hay excepcion>
DESPUES nombre=... hambre=... energia=... felicidad=... color=... comidas=...
PUEDE_JUGAR <true o false; obligatorio en T1-T4>
```

Los puntos y marcadores son una plantilla: sustituidlos por los valores observados.

Compilar, ejecutar y conservar la evidencia

Desde la carpeta que contiene `pom.xml`, compilad vuestro archivo junto al proyecto:

```
# macOS / Linux
./mvnw compile
# Windows PowerShell
.\mvnw.cmd compile
```

Después ejecutad **vuestra clase**, con el mismo comando en los tres sistemas:

```
java -cp target/classes es.uv.is2.mascota.PruebasCajaNegra
```

Cuando se hayan ejecutado todas las filas, repetid guardando la salida:

```
java -cp target/classes es.uv.is2.mascota.PruebasCajaNegra > ejercicios/caja-negra.txt
```

GitHub: debe actualizarse para incorporar el fichero de salida `caja-negra.txt`.

Revisión de las pruebas de caja negra

Revisión de evidencias. Responsable y revisor comprobarán que el código ejecuta las 23 filas y que las tablas reflejan la salida real.

3. Pruebas de caja blanca

Prueba de caja blanca guiada

Analizad el fragmento de `ejercicios/caja-blanca/ReglaJuego.java`. Su contrato exige hambre ≤ 80 y energía ≥ 20; debéis contrastar si el código lo cumple. Es independiente de la mascota y no debéis copiarlo sobre sus clases.

```java
public class ReglaJuego {
    public static boolean puedeJugar(int hambre, int energia) {
        boolean hambrePermitida = hambre <= 80;
        boolean energiaSuficiente = energia >= 20;
        if (hambrePermitida || energiaSuficiente) {
            return true;
        }
        return false;
    }

    public static void main(String[] args) {
        if (args.length != 2) {
            throw new IllegalArgumentException("Uso: java ReglaJuego.java HAMBRE ENERGIA");
        }
        System.out.println(puedeJugar(Integer.parseInt(args[0]), Integer.parseInt(args[1])));
    }
}
```

Pasos a seguir:

1. Marcad las sentencias ejecutadas al usar hambre 50 y energía 50. Identificad qué sentencia ejecutable queda sin recorrer. No contéis llaves ni la cabecera del método como sentencias.
2. Añadid un caso para obtener los dos resultados de la decisión del `if`. Justificad la cobertura de sentencias y de decisión alcanzada por esos dos casos.
3. Recuperad las cuatro combinaciones de vuestra tabla de decisión (ver apartado 2 - Tabla 1). Para cada una, anotad el valor de las dos condiciones simples y de la decisión completa.
4. Explicad si haber obtenido una decisión verdadera y otra falsa demuestra que cada condición se ha evaluado como verdadera y falsa. Considerad el cortocircuito de `||`: si el primer operando es verdadero, no se evalúa el segundo operando dentro del `if`, aunque la variable ya se haya calculado antes.
5. Comparad los dos casos mínimos del paso 2 con las cuatro combinaciones del paso 3. Contrastad los resultados con el contrato: identificad una combinación que revele un incumplimiento y explicad por qué cubrir ambas decisiones puede dejarlo sin descubrir. Ejecutad el fragmento para comprobar vuestra traza.
6. Corregid `ejercicios/caja-blanca/ReglaJuego.java` para que las pruebas obtengan el resultado correcto. Documentad la corrección en la memoria.

Desde la raíz, ejecutad `java ejercicios/caja-blanca/ReglaJuego.java 81 50`; cambiad los dos números para probar el resto de combinaciones y rellenad la tabla que sigue apropiadamente:

Caso	Hambre / energía	Condición hambre	Condición energía en el if	Decisión / retorno
CB1				
CB2				
CB3				
CB4				

Memoria: incluid la secuencia de pruebas y los resultados en la tabla. Diferenciad “falso” de “no evaluado” por cortocircuito.

Diseñar e implementar pruebas

Crearéis y trabajaréis ahora con `ejercicios/caja-blanca/ReglaDescanso.java`. Es otro fragmento independiente: no lo copiéis sobre `ServicioDescanso` ni sobre la mascota.

Contrato. La entrada es un entero entre 0 y 100. Descansar aumenta la energía en 30, sin superar 100. Si la energía ya es 100, el método lanza `IllegalArgumentException`. En este ejercicio no se piden pruebas fuera de la precondición ni se modifica una mascota: se comprueba el valor devuelto o la excepción.

```
public class ReglaDescanso {
    // Precondicion: energia es un entero entre 0 y 100.
    public static int energiaTrasDescanso(int energia) {
        if (energia == 100) {
            throw new IllegalArgumentException("No necesita descansar");
        }
        int resultado = energia + 30;
        if (resultado > 100) {
            return 99;
        }
        return resultado;
    }
}
```

Pasos a seguir:

1. Identificad los caminos desde la entrada hasta cada retorno o excepción. Describid las decisiones recorridas y señalad cuándo la segunda condición no llega a evaluarse.
2. Elegid vuestras propias entradas para recorrer todos los caminos. Añadid un caso cuyo incremento produzca exactamente el máximo permitido. Justificad qué aporta este caso respecto a otro del mismo camino. No se proporcionan entradas resueltas.
3. Antes de ejecutar, calculad las expectativas desde el contrato. Cread `PruebasDescanso.java` en la carpeta del ejercicio, sin declaración de paquete y con un `main`. Llamad al método estático `ReglaDescanso.energiaTrasDescanso(entrada)` para cada caso.
4. Implementad la comparación del valor devuelto con el esperado y comprobad los rechazos mediante `try/catch` de `IllegalArgumentException`. Imprimid ID, entrada, expectativa, observación y C/F. Un rechazo inesperado, un retorno cuando se esperaba rechazo o un valor distinto deben producir F. Continuad hasta ejecutar todos los casos.
5. Conservad la salida inicial y localizad el incumplimiento, si lo hay.
   Para la ejecución, desde la raíz del proyecto, compilad el fragmento y vuestro ejecutor. El compilador creará la carpeta de salida; estos comandos son iguales en macOS, Linux y Windows PowerShell:

   ```
   javac -d target/caja-blanca ejercicios/caja-blanca/ReglaDescanso.java ejercicios/caja
       -blanca/PruebasDescanso.java
   java -cp target/caja-blanca PruebasDescanso > ejercicios/caja-blanca/caja-blanca-
       descanso-inicial.txt
   ```

   GitHub: debe actualizarse para incorporar el fichero de salida `caja-blanca-descanso-inicial.txt`.
6. Corregid el fragmento, recompilad y repetid **las mismas pruebas y expectativas**. No cambiéis los valores esperados para hacer coincidir las pruebas con el código.
   Después de corregir `ReglaDescanso.java`, repetid el comando de compilación y guardad la nueva salida en otro archivo:

   ```
   javac -d target/caja-blanca ejercicios/caja-blanca/ReglaDescanso.java ejercicios/caja
       -blanca/PruebasDescanso.java
   java -cp target/caja-blanca PruebasDescanso > ejercicios/caja-blanca/caja-blanca-
   ```

```
descanso-corregido.txt
```

No sobrescribáis el registro inicial. Si el ejecutor termina con código de error por pruebas fallidas, comprobad sus resultados: un fallo de aserción previsto no es lo mismo que un error de compilación o de arranque.
GitHub: debe actualizarse para incorporar el fichero de salida `caja-blanca-descanso-corregido.txt`.

7. Explicad por qué recorrer todos los caminos no garantiza detectar un fallo si no se comprueban los resultados. Distinguid la cobertura alcanzada del veredicto de las pruebas.

Memoria: detallad los pasos seguidos y las conclusiones obtenidas. Podéis apoyaros en una tabla como la que sigue para presentar los resultados:

ID	Entrada	Camino	Esperado	Observado inicial	Veredicto inicial	Observado corregido	Veredicto
CD1							
CD2							
CD3							
CD4							

GitHub: debe actualizarse para incorporar las implementaciones de este apartado (`ReglaDescanso.java` y `PruebasDescanso.java`), además de cualquier otro auxiliar creado en el contexto de este ejercicio y los dos ficheros de salida.

Revisión de las pruebas de caja blanca

Revisión de evidencias. Responsable y revisor comprobarán que el código ejecuta las pruebas esperadas y que las tablas reflejan la salida real.

4. Automatizar con JUnit

La configuración de JUnit se proporciona en el proyecto.

Ejemplo guiado

En `src/test/java/es/uv/is2/mascota/`, cread el archivo `PR3JuegoTest.java`. Este ejemplo comprueba un juego válido y un rechazo:

```
package es.uv.is2.mascota;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import static org.junit.jupiter.api.Assertions.*;

class PR3JuegoTest {
    @Test
    void jugarActualizaLosTresNiveles() {
        Mascota m = new Mascota("Luna", 50, 50, 50);
        ServicioJuego servicio = new ServicioJuego(m);
        servicio.jugar();
        assertEquals(60, m.getHambre());
        assertEquals(30, m.getEnergia());
        assertEquals(65, m.getFelicidad());
    }

    @Test
    void rechazarNoCambiaElEstado() {
        Mascota m = new Mascota("Luna", 50, 19, 50);
        ServicioJuego servicio = new ServicioJuego(m);
        assertThrows(IllegalArgumentException.class,
                     () -> servicio.jugar());
        assertEquals(50, m.getHambre());
        assertEquals(19, m.getEnergia());
        assertEquals(50, m.getFelicidad());
    }

    @ParameterizedTest
    @CsvSource({"50, 50, true", "81, 50, false"})
    void permisoDeJuego(int hambre, int energia,
                        boolean esperado) {
        Mascota m = new Mascota("Luna", hambre, energia, 50);
        assertEquals(esperado, m.puedeJugar());
    }
}
```

- `@Test` identifica una prueba.
- `assertEquals(esperado, real)` compara valores.
- `assertThrows` comprueba la excepción indicada. En la prueba parametrizada, cada fila de
- `@CsvSource` provoca una ejecución con otros argumentos.
- La expresión `() -> servicio.jugar()` indica la operación que JUnit debe ejecutar para comprobar el rechazo.

Ejecución

Desde la carpeta que contiene `pom.xml`, ejecutad la clase anterior:

```
# macOS / Linux
./mvnw -Dtest=PR3JuegoTest test
# Windows PowerShell
.\mvnw.cmd -Dtest=PR3JuegoTest test
```

Usad solo el comando de vuestro sistema. **Las cuatro ejecuciones del ejemplo guiado deben pasar también en la versión inicial con defectos**: una corresponde al juego válido, otra al rechazo sin cambios y dos a las filas de la prueba parametrizada. Si habéis copiado el ejemplo, el resultado esperado es:

```
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Este resultado confirma que el ejemplo se ejecuta correctamente, pero sus casos todavía no descubren los defectos preparados.

Los fallos previstos aparecerán al ampliar las pruebas en el siguiente bloque, en particular con energía exactamente 20 y con felicidad inicial 95 para comprobar la saturación. Sobre la versión inicial, esas pruebas deben revelar incumplimientos del contrato. Identificad entonces el test fallido, la expectativa y el resultado observado. Un fallo de aserción puede ser la evidencia buscada; un error de compilación no demuestra que se haya detectado un defecto.

Ampliación de la automatización

Pasos a seguir:

1. Ampliad la parametrizada con las cuatro reglas de decisión y los seis casos frontera del apartado 2. Podéis reunir casos coincidentes en una misma fila si conserváis la trazabilidad.
2. Añadid una prueba del juego permitido justo con hambre 80 y energía 20. Verificad los tres niveles finales. Añadid otra con felicidad 95 para comprobar la saturación.
3. Ampliad la prueba de rechazo para comprobar también nombre, color y comidas. Cubrid por separado el rechazo por hambre excesiva y por energía insuficiente.
4. En `PR3NombreTest.java`, automatizad un nombre válido con espacios exteriores y un nombre de 21 letras rechazado sin cambiar el anterior. Usad `new ServicioAspecto(m).renombrar(nombre)` y `m.getNombre()`.

Cada test debe crear sus propios objetos y comprobar el comportamiento previsto sin depender de otro test.

En este caso deben observarse errores al ejecutar el test, viéndose algo como:

```
[ERROR] Tests run: 20, Failures: 5, Errors: 1, Skipped: 0
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
```

Memoria: elaborad una tabla donde se detallen qué test automáticos (implementados con JUnit) se han ejecutado y sus resultados.

GitHub: actualizad el repositorio al terminar las implementaciones elaboradas para este apartado.

Revisión de la automatización

Revisión de evidencias. Responsable y revisor comprobarán que la automatización implementada ejecuta las pruebas esperadas y que la tabla refleja la salida real.

5. Probar la integración

En la automatización del apartado 4 se llama directamente a `ServicioJuego`. En cambio, en esta sección comprobaremos que **servicio, controlador e historial colaboran (se integran) correctamente**: una acción válida debe modificar la mascota y dejar un único evento con el estado posterior; una acción rechazada no debe modificarla ni dejar eventos.

Para esta prueba de integración se usan los componentes reales, sin sustitutos ni ventana gráfica. Incorporar colaboradores a partir del servicio ya probado aplica una estrategia ascendente. El test JUnit actúa como **conductor**: construye los objetos, inicia la acción y comprueba sus efectos.

- `ServicioJuego` comprueba si se puede jugar y modifica la mascota.
- `ControladorMascota` recibe la acción que se quiere ejecutar y coordina su ejecución y registro.
- `RegistroAcciones` guarda objetos `Evento`, con una copia de los valores de la mascota. El evento es una fotografía del momento del registro, no una consulta al estado actual.

Preparar dos tests independientes

Cread `PR3IntegracionTest.java` en la misma carpeta de tests del apartado 4, con el paquete `es.uv.is2.mascota`, el import de `Test` y los imports estáticos de las aserciones JUnit. Implementad dos métodos con `@Test`: uno para I1 (acción válida) y otro para I2 (rechazo). No necesitan `main`.

En **cada test** cread una mascota, un registro vacío, un controlador y un servicio nuevos. Controlador y servicio deben recibir **la misma instancia** de mascota, y el controlador debe recibir el registro que después comprobaréis.

Para I1, la preparación es:

```
Mascota m = new Mascota("Luna", 50, 50, 50);
RegistroAcciones registro = new RegistroAcciones();
ControladorMascota controlador = new ControladorMascota(m, registro);
ServicioJuego juego = new ServicioJuego(m);
```

Para I2, repetid la preparación dentro de otro test usando energía 19. Comprobad que el registro comienza vacío. No reutilicéis objetos de otro test: el resultado no debe depender del orden de ejecución.

I1. Una acción válida debe registrarse correctamente

Ejecutad **una sola vez**, después de preparar los objetos:

```
controlador.ejecutar(Accion.JUGAR, () -> juego.jugar());
```

`Accion.JUGAR` identifica el evento. La expresión `() -> juego.jugar()` entrega al controlador la operación que debe invocar. No llaméis además a `juego.jugar()` antes o después: estaríais realizando dos acciones.

Comprobad con aserciones: estado final 60/30/65, nombre/color/comidas conservados, exactamente un evento, acción JUGAR y estado 60/30/65 en ese evento. **Comprobad primero el tamaño** con `registro.eventos().size()`; solo después acceded a `registro.eventos().get(0)`. El evento ofrece `accion()`, `hambre()`, `energia()` y `felicidad()`, sin prefijo `get`. Usad valores esperados explícitos: comparar solo evento y mascota podría aceptar dos resultados igualmente incorrectos.

I2. Una acción rechazada no debe dejar efectos

Preparad Luna con 50/19/50 y un registro nuevo. Guardad también nombre, color y comidas antes de la llamada. Comprobad el rechazo del **recorrido completo**, no solo del servicio:

```
assertThrows(IllegalArgumentException.class,
    () -> controlador.ejecutar(Accion.JUGAR, () -> juego.jugar()));
```

JUnit ejecuta la llamada y verifica que lanza esa excepción. Si se cumple, el test continúa después de `assertThrows`: añadid las aserciones que comprueben que H/E/F sigue siendo 50/19/50, nombre/color/comidas se conservan y `registro.eventos().isEmpty()` es verdadero. **Una excepción correcta no garantiza que no haya efectos secundarios.**

Ejecutar, observar e interpretar

Desde la carpeta de `pom.xml`, ejecutad únicamente vuestra clase de integración:

```
# macOS / Linux
./mvnw -Dtest=PR3IntegracionTest test
# Windows PowerShell
.\mvnw.cmd -Dtest=PR3IntegracionTest test
```

El resumen esperado es 2 pruebas, 2 fallos y `BUILD FAILURE`.

Consultad la salida y los informes de `target/surefire-reports/`. Registrad qué aserción falla y sus valores esperado y real. Podéis agrupar aserciones independientes con `assertAll` o inspeccionar los objetos con el depurador. Si una aserción impide llegar a otra, no inventéis la observación posterior: obtenedla al inspeccionar los objetos o indicad que no se comprobó.

Memoria: documentad los pasos seguidos para este test de integración y los resultados obtenidos. Podéis apoyaros en completar la tabla siguiente.

Tabla de evidencias. Son dos tests con varias comprobaciones, no siete tests. IAE abrevia `IllegalArgumentException`. Completad observado y C/F; si no podéis comprobar una fila, explicad la causa en la memoria.

Caso	Comprobación	Esperado	Observado	C/F
I1	Estado H/E/F	60/30/65		
I1	Número de eventos	1		
I1	Acción del evento	JUGAR		
I1	H/E/F del evento	60/30/65		
I2	Excepción	IAE		
I2	Estado H/E/F	50/19/50		
I2	Número de eventos	0		

Anotad también la conservación de nombre, color y comidas en ambos casos. Si falla la integración, conservad la evidencia: no añadáis ni borréis eventos desde el test, no rebajéis las expectativas y no sustituyáis los componentes reales. En este apartado se pide detectar y explicar; no es hay que corregir el controlador para continuar con el apartado 6.

Memoria: explicad qué colaboración se ha comprobado y qué fallo podría escapar si solo se probase el servicio.

GitHub: subid al repositorio los dos tests y la salida/informe de JUnit que sustente los resultados.

Revisión. Responsable y revisor comprobarán que cada test crea sus objetos, ejecuta una sola acción a través del controlador y verifica mascota e historial.

6. Depurar un defecto y conservar la regresión

Un usuario comunica el siguiente problema: *«La mascota tiene toda la energía. Intento descansar, aparece un rechazo, pero después veo que el hambre ha cambiado»*. En este apartado investigaréis ese fallo hasta explicar su causa y corregirla. Conservaréis además una **prueba de regresión**: un test que vuelva a detectar el defecto si una modificación futura lo introduce de nuevo.

El contrato del descanso es el siguiente: si la energía es menor que 100, la acción aumenta el hambre en 5 y la energía en 30, sin superar 100; la felicidad no cambia. Si la energía ya es 100, debe lanzar la excepción `IllegalArgumentException` y conservar el estado completo. Nombre, color y comidas no cambian en ningún caso. Por tanto, mostrar un rechazo no basta: también hay que comprobar que la mascota queda como estaba.

Trabajaréis con `ServicioDescanso`, el servicio real de la aplicación. El fragmento `ReglaDescanso` del apartado 3 es un ejercicio independiente. Empezaréis llamando directamente al servicio, sin controlador, historial ni interfaz, para investigar el caso con los mínimos componentes necesarios.

En la sección 6.6 se describe qué debe figurar en la memoria respecto a este apartado. Leedlo antes de seguir los pasos descritos en el resto de subsecciones del apartado.

6.1. Reproducir el problema y guardar el estado inicial

Antes de editar el servicio, identificad la versión mediante `git rev-parse HEAD` y guardad los cambios locales que muestren los comandos `git status --short` y `git diff`. Así podréis volver al estado que produjo el fallo, incluso si había cambios sin confirmar.

Usaréis tres casos, siempre con una mascota nueva llamada Luna:

- **R1** reproduce el aviso con hambre/energía/felicidad (H/E/F) = 50/100/50 y debe rechazarse sin cambios.
- **R2** parte de 50/40/50 y debe aceptar el descanso, terminando en 55/70/50.
- **R3** parte de 98/90/50 y debe terminar en 100/100/50: comprueba que los incrementos se limitan a 100.

Los dos últimos casos permitirán comprobar después que el arreglo conserva el descanso permitido.

Desde la carpeta de `pom.xml`, compilad y ejecutad los tres casos en este orden:

```
# macOS / Linux
./mvnw compile
# Windows PowerShell
.\mvnw.cmd compile
# Todos los sistemas: R1, R2 y R3
java -cp target/classes es.uv.is2.mascota.AplicacionMascota --caso descansar 50 100 50
java -cp target/classes es.uv.is2.mascota.AplicacionMascota --caso descansar 50 40 50
java -cp target/classes es.uv.is2.mascota.AplicacionMascota --caso descansar 98 90 50
```

La consola muestra el estado anterior, la aceptación o el rechazo y el estado posterior. Conservad las tres salidas para la columna *Antes de corregir* de la tabla final (ver apartado 6.6, donde se describe la tabla a incluir en la memoria). Estos comandos usan las clases recién compiladas; `compile` no ejecuta los tests, por lo que podéis reproducir el caso aunque queden pruebas fallidas de otros apartados.

6.2. Escribir la prueba que debe fallar

Una vez observado el problema, convertid R1 en una comprobación automática. Esta prueba expresará lo que exige el contrato y deberá fallar con la versión inicial. Cread `PR3DescansoTest.java` en `src/test/java/es/uv/is2/mascota/`, con el paquete `es.uv.is2.mascota` y los imports JUnit utilizados en el apartado 4. El test debe seguir esta secuencia:

1. Cread una mascota nueva con 50/100/50 y guardad nombre, color y comidas iniciales. Construid `ServicioDescanso` con esa mascota.
2. Comprobad mediante `assertThrows` que la llamada directa a `servicio.descansar()` lanza `IllegalArgumentException`. No uséis el controlador en este test.
3. **Después de comprobar la excepción**, añadid aserciones para verificar que H/E/F sigue siendo 50/100/50 y que los otros campos se conservan. Podéis agruparlas con `assertAll`. Capturar el rechazo sin comprobar el estado no detecta este fallo.

Ejecutad solo esa clase y conservad su resultado **antes de corregir**:

```
# macOS / Linux
./mvnw -Dtest=PR3DescansoTest test
# Windows PowerShell
.\mvnw.cmd -Dtest=PR3DescansoTest test
```

Debe fallar una aserción sobre el estado observado (un error de compilación, una clase no encontrada o una prueba no ejecutada no demuestran el defecto). Si el test pasa inicialmente, revisad el objeto utilizado, las aserciones y la versión probada; no cambiéis el contrato para obtener un resultado concreto.

6.3. Formular una hipótesis y contrastarla

El test demuestra que hay una diferencia respecto al contrato, pero todavía debéis averiguar qué la produce. Antes de cambiar código, escribid una explicación que prediga algo observable (en la memoria debe figurar vuestra hipótesis de lo que ocurre y la predicción de los valores que podrán observarse durante la ejecución). Por ejemplo: "*alguna instrucción modifica la mascota aunque la acción termina rechazándose; si es así, podré localizar el primer cambio de valor durante la llamada*". No basta decir "*el método está mal*" ni dar por cierta una causa porque parece plausible.

Para contrastarla en VS Code:

1. Abrid `ServicioDescanso.java` y colocad un punto de interrupción en la primera instrucción de `descansar()`. Iniciad **Debug Test** sobre vuestro test R1.
2. Al detenerse, inspeccionad `mascota`: hambre, energía y felicidad. Registrad los valores de entrada. Usad **Step Over** para avanzar instrucción a instrucción y anotad el primer punto en que cambia alguno.
3. Observad qué valores hay cuando se decide rechazar y qué instrucción lanza la excepción. Contrastad esos hechos con vuestra predicción. No cambiéis variables desde el depurador: alteraríais el experimento.
4. Continuad la ejecución: JUnit debe capturar la excepción y ejecutar las aserciones posteriores. Explicad qué observación apoya la hipótesis o la refuta. Si no coincide, revisad la hipótesis y repetid el contraste.

Si usáis trazas temporales en vez del depurador, imprimid los valores antes y después de las instrucciones relevantes y junto al rechazo. Conservad la evidencia y retirad las trazas al terminar. Que el fallo se reproduzca llamando solo al servicio demuestra que no hace falta el historial para provocarlo; no implica que el resto del sistema esté libre de defectos.

6.4. Corregir el mecanismo y repetir la misma prueba

A partir de las observaciones del paso anterior, explicad qué instrucción o secuencia causa el cambio indebido y por qué. Aplicad el cambio mínimo que permita rechazar sin efectos secundarios y conserve el descanso válido. No silenciéis la excepción, no cambiéis los valores esperados del test y no eliminéis la actualización necesaria para los casos permitidos. Revisad el diff y comprobad que está limitado al defecto investigado.

Repetid el comando de `PR3DescansoTest` del paso 2: Maven recompila el servicio modificado. El **mismo test** que fallaba debe pasar ahora. Conservad las dos ejecuciones por separado. Si no pasa, volved a las observaciones: editar código no prueba por sí solo que la causa esté resuelta.

En la memoria debe documentarse el defecto y su corrección.

6.5. Comprobar que el descanso válido sigue funcionando

Que R1 pase demuestra que ese rechazo conserva el estado, pero no garantiza que siga siendo posible descansar cuando corresponde. Por eso, ahora debéis repetir los tres comandos del paso 6.1 sobre las clases recompiladas por Maven y comparar las nuevas salidas con las iniciales.

R1 debe rechazar sin modificar la mascota; R2 debe seguir realizando el descanso habitual y R3 debe seguir limitando los niveles a 100. Registrad estos resultados en la columna *Después* de la tabla (ver sección 6.6; incluir esto en la memoria).

6.6. Revisar la suite y documentar la regresión

Tras comprobar el descanso de forma aislada, ejecutad el conjunto completo de pruebas para detectar si el cambio ha afectado a otros comportamientos:

```
# macOS / Linux
./mvnw test
# Windows PowerShell
.\mvnw.cmd test
```

Puede seguir habiendo fallos de juego, nombres, integración o interfaz documentados en otros apartados. Identificad qué tests siguen fallando y relacionadlos con defectos pendientes (dejad constancia de esto en la memoria). Un resultado global rojo no demuestra que R1 siga fallando, y un R1 verde no demuestra que toda la aplicación esté corregida. Si aparece un fallo nuevo, investigad su relación con el cambio antes de dar la corrección por válida.

Tabla de comprobación del descanso (debe incluirse en la memoria entregada). En cada observación anotad aceptación/rechazo, estado final y C/F frente al contrato. En todos los casos se deben conservar nombre, color y comidas. R1 es el test automatizado de regresión; R2 y R3 son controles del comportamiento válido.

Caso	H/E/F inicial	Esperado	Antes de corregir	Después
R1	50/100/50	Rechaza; 50/100/50.		
R2	50/40/50	Acepta; 55/70/50.		
R3	98/90/50	Acepta; 100/100/50.		

GitHub: subid la prueba de regresión y la corrección.

Revisión. Responsable y revisor deben poder explicar qué fallaba, qué observación localiza la causa, por qué el cambio la corrige y qué prueba detectará su reaparición.

7. Comprobar la sensibilidad mediante mutación

En los apartados anteriores habéis usado pruebas para descubrir defectos del programa. Para comprobar si esas pruebas serían capaces de detectar ciertos cambios incorrectos, ejecutaréis una misma suite sobre una implementación correcta y sobre dos versiones que contienen una modificación artificial, llamadas **mutantes**. Si un test pasa con la original y falla con un mutante por el comportamiento modificado, lo ha detectado. Si todos los tests pasan también con el mutante, este **sobrevive a esa suite**: las pruebas utilizadas no distinguen ambas versiones.

El ejercicio se encuentra en `ejercicios/mutacion/` y es independiente de la aplicación. Tiene su propio `pom.xml`, una clase `ReglaJuego` y sus tests. El método `permite(hambre, energia)` devuelve un booleano: debe devolver `true` exactamente cuando hambre ≤ 80 **y** energía ≥ 20; en cualquier otro caso, `false`. No modifica una mascota ni lanza una excepción por denegar el juego. Este contrato será vuestro criterio para decidir las expectativas de todos los tests.

7.1. Entender qué vais a comparar

Dentro del ejercicio hay tres carpetas de código fuente. Cada una contiene `ReglaJuego.java` en la subcarpeta `java/es/uv/is2/mutacion/`:

- `src/original/`: la implementación correcta usa `hambre <= 80 && energia >= 20`.
- `src/frontera/`: cambia únicamente `energia >= 20` por `energia > 20`.
- `src/condiciones/`: cambia únicamente `&&` por `||`, de modo que basta cumplir una de las dos condiciones.

Cada mutante parte de la original: los cambios no se acumulan. Ya están preparados y no debéis corregirlos. Vuestra tarea será mejorar las pruebas para que revelen la diferencia. Es mutación manual controlada con JUnit.

Abrid el test del ejercicio, cuya ruta desde la raíz de la mascota es:

```
ejercicios/mutacion/src/test/java/es/uv/is2/mutacion/ReglaJuegoTest.java
```

Contiene un test parametrizado con `@CsvSource`: cada fila proporciona hambre, energía y resultado esperado. Las dos filas iniciales son `"50, 50, true"` y `"81, 19, false"`. Aunque hay un único método de test, se ejecuta una vez por fila.

7.2. Ejecutar y conservar los resultados iniciales

Desde la carpeta principal de la mascota, donde está `mvnw`, ejecutad estos comandos **por separado**:

```
./mvnw -f ejercicios/mutacion/pom.xml -Dvariante=original clean test
./mvnw -f ejercicios/mutacion/pom.xml -Dvariante=frontera clean test
./mvnw -f ejercicios/mutacion/pom.xml -Dvariante=condiciones clean test
```

En Windows PowerShell, sustituid `./mvnw` por `.\mvnw.cmd`. La opción `-f` selecciona el proyecto del ejercicio y `-Dvariante` elige una de las tres implementaciones. En los tres casos se ejecutan los mismos tests; no se ejecuta la suite de la aplicación principal.

En la primera ejecución deben pasar las tres variantes. Para cada ejecución, guardad el resumen de JUnit y, si hay fallos, la aserción y los valores esperado y real. Los informes están dentro de `ejercicios/mutacion/target/`: abrid la carpeta de la variante y después `surefire-reports/`. Copiad las evidencias iniciales en la memoria antes de repetir: `clean` borra la salida anterior de la variante seleccionada.

7.3. Ampliar las pruebas para detectar los dos mutantes

Las dos entradas iniciales no distinguen las implementaciones: con 50/50 se cumplen ambos requisitos y con 81/19 no se cumple ninguno. Por eso las tres variantes pasan dos pruebas en el apartado 7.2. Ahora debéis **conservar esos dos casos y añadir los tres siguientes**, para comprobar la frontera y las condiciones por separado:

1. **Hambre 50, energía 20: frontera incluida.** La condición de hambre se cumple. Comparad qué ocurre exactamente en energía 20 con `>=` y con `>`. ¿Qué resultado exige el contrato y qué variante responderá de otra forma?
2. **Hambre 81, energía 50: solo se cumple la condición de energía.** Decidid si cumplir ese único requisito basta según el contrato. Comparad la respuesta de una conjunción con la de una disyunción.
3. **Hambre 50, energía 19: solo se cumple la condición de hambre.** Repetid el razonamiento con el otro requisito incumplido. Este caso comprueba la otra combinación mixta.

Memoria: antes de ejecutar, anotad para cada entrada el booleano exigido por el contrato y el que predecís para cada una de las tres implementaciones. Justificad las diferencias a partir de los operadores.

Modificad únicamente el archivo de pruebas del ejercicio:

```
ejercicios/mutacion/src/test/java/es/uv/is2/mutacion/ReglaJuegoTest.java
```

Dentro de `@CsvSource`, conservad las dos filas existentes y añadid una por cada entrada anterior. Cada fila contiene `hambre, energia, esperado`, entre comillas y separada de las demás por una coma; sustituid `esperado` por el booleano que habéis deducido del contrato. Al terminar debe haber **cinco filas**. Conservad `@ParameterizedTest`, el método y su aserción `assertEquals`; no hace falta crear tres métodos ni una clase diferente para cada variante.

Las mismas cinco expectativas se utilizan con las tres implementaciones. **No modifiquéis las clases originales o mutantes ni adaptéis las expectativas para que todas pasen.** El objetivo es que las pruebas detecten los cambios incorrectos.

Guardad el archivo y repetid, por separado, los tres comandos de 7.2: primero `original`, después `frontera` y finalmente `condiciones`. Aunque Maven muestre un fallo en una variante, ejecutad también la siguiente. Conservad las tres salidas de la suite ampliada para compararlas con las iniciales.

7.4. Interpretar y documentar el resultado

Cada fila del test parametrizado cuenta como una ejecución. Por tanto, ahora deben ejecutarse **cinco pruebas en cada variante**. Con los casos anteriores, el resultado esperado es:

- `original`: las cinco pruebas pasan, con `BUILD SUCCESS`.
- `frontera`: cuatro pasan y una falla, con `BUILD FAILURE`. Localizad la entrada que comprueba el límite de energía.
- `condiciones`: tres pasan y dos fallan, con `BUILD FAILURE`. Localizad las dos entradas en las que solo se cumple uno de los requisitos.

En los tres resúmenes se espera `Tests run: 5`, `Errors: 0` y `Skipped: 0`; `Failures` debe ser 0, 1 y 2, respectivamente. **Los dos mutantes deben quedar detectados.** Sus fallos son el resultado buscado: muestran que las nuevas pruebas distinguen cada cambio artificial de la implementación correcta.

No basta con copiar el resumen: para cada fallo, identificad la entrada, el valor esperado, el observado y el operador que explica la diferencia. Contrastadlos con vuestra predicción de 7.3. Si un mutante sigue pasando, comprobad que se han añadido las cinco filas y se ha ejecutado la variante correcta. Si falla la original, revisad las expectativas y el código del test. Un error de compilación, una dependencia

ausente o una prueba no ejecutada no acreditan detección.

Memoria: completad ambas tablas con vuestras salidas reales. En la primera registrad cuántos tests pasan y fallan antes y después; para cada mutante, indicad también si sobrevive o se detecta. Debéis conservar las seis ejecuciones: tres iniciales y tres con la suite ampliada.

Variante	Suite inicial	Suite reforzada
original		
frontera		
condiciones		

En la segunda tabla registrad el booleano exigido por el contrato y los observados en la original y en el mutante indicado. Cada fila debe corresponder a una aserción que pasa con la original y falla con ese mutante.

Mutante	H/E	Contrato	Original	Mutante
frontera	50/20			
condiciones	81/50			
condiciones	50/19			

Memoria: explicad por qué las dos entradas iniciales dejaban sobrevivir ambos mutantes y qué aporta cada uno de los tres casos añadidos. Razonad por qué ejecutar una línea no garantiza detectar un resultado incorrecto.

GitHub: subid el test ampliado con las cinco filas.

Revisión. Responsable y revisor comprobarán que la original pasa, ambos mutantes se detectan y cada aserción fallida corresponde al cambio estudiado.

8. Aceptación y decisión final

Ejecutad la aplicación completa desde una instancia nueva y realizad el escenario siguiente. La persona revisora asumirá el papel de usuario con esta necesidad: *cuidar de la mascota y distinguir las acciones realizadas de los intentos rechazados.*

Para generar y abrir el programa, usad el comando de construcción de vuestro sistema y después el comando de Java:

```
# macOS / Linux
./mvnw package
# Windows PowerShell
.\mvnw.cmd package
# Ambos sistemas, tras una construccion correcta
java -jar target/mascota-1.0.0.jar
```

A continuación, probad la aplicación con los pasos descritos en la tabla que sigue:

Paso	**Criterio de aceptación observable**
Inicio nuevo	Hambre, energía y felicidad 50/50/50; historial vacío.
Jugar dos veces	Estado 70/10/80 y dos eventos de juego.
Intentar jugar otra vez	Mensaje de rechazo; se conservan 70/10/80 y dos eventos.
Descansar	Estado 75/40/80 y tercer evento de descanso.
Cambiar nombre a Nube	Se muestra Nube; niveles conservados y cuarto evento de cambio de nombre.

Memoria:

1. Registrad qué criterios se cumplen. Si un paso falla, conservad la evidencia e identificad qué pasos posteriores quedan bloqueados. Comprobad además el renombrado en una instancia nueva: debe verse Nube. Registrad ese ensayo independiente sin atribuirle el éxito del recorrido completo.
2. La persona que actúa como usuario comprobará si entiende por qué se ha rechazado el juego y si el historial le permite distinguir acciones realizadas e intentos. Anotad una observación sobre la utilidad o claridad del resultado.
3. Acordad **entregar o detener la entrega**. Justificad la decisión con las pruebas ejecutadas, defectos pendientes y limitaciones.